UNIDAD 1.

SELECCIÓN DE ARQUITECTURAS Y HERRAMIENTAS A UTILIZAR

DESARROLLO WEB EN ENTORNO CLIENTE

Autor: **Ramón Javier Morales Pacheco**

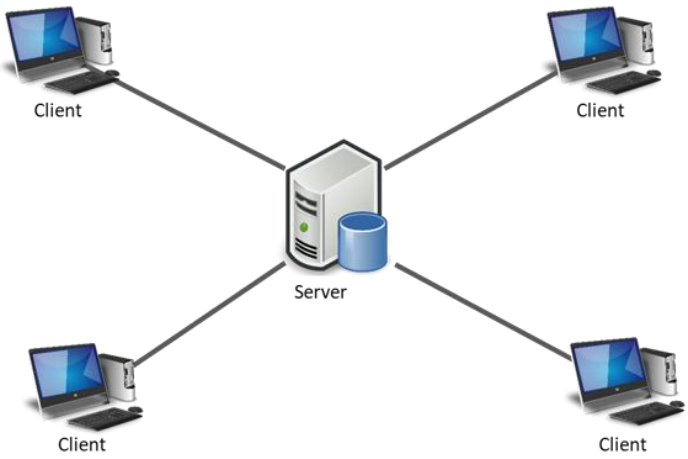
INDICE

1. Modelos de programación en entornos cliente/servidor.
   1.1. Arquitectura Cliente-Servidor.
2. Navegadores.
3. Funcionamiento de los navegadores.
4. Compatibilidad con navegadores Web.
5. Lenguajes de programación en entorno cliente.
6. Características de los lenguajes de script. Ventajas y desventajas sobre la programación tradicional.
7. Tecnologías y lenguajes asociados.
8. Integración del código con las etiquetas HTML.
9. Herramientas de programación.

1. Modelos de programación en entornos cliente/servidor.

Cliente-Servidor es uno de los estilos arquitectónicos distribuidos más conocidos, el cual está compuesto por dos componentes, el proveedor y el consumidor. El proveedor es un servidor que brinda una serie de servicios o recursos los cuales son consumidos por el Cliente.

En una arquitectura Cliente-Servidor existe un servidor y múltiples clientes que se conectan al servidor para recuperar todos los recursos necesarios para funcionar, en este sentido, el cliente solo es una capa para representar los datos y se detonan acciones para modificar el estado del servidor, mientras que el servidor es el que hace todo el trabajo pesado.



En esta arquitectura, el servidor deberá exponer un mecanismo que permite a los clientes conectarse, que por lo general es TCP/IP, esta comunicación permitirá una comunicación continua y bidireccional, de tal forma que el cliente puede enviar y recibir datos del servidor y viceversa.

Creo que es bastante obvio decir que en esta arquitectura el cliente no sirve para absolutamente nada si el servidor no está disponible, mientras que el servidor por sí solo no tendría motivo de ser, pues no habría nadie que lo utilice. En este sentido, las dos partes son mutuamente dependientes, pues una sin la otra no tendría motivo de ser.

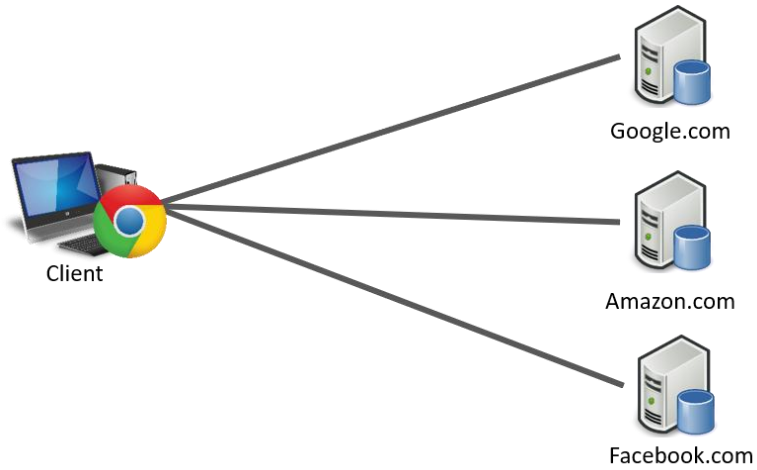
Cliente-Servidor es considerada una arquitectura distribuida debido a que el servidor y el cliente se encuentran distribuidos en diferentes equipos (aunque podrían estar en la misma máquina) y se comunican únicamente por medio de la RED o Internet.

En esta arquitectura, el Cliente y el Servidor son desarrollados como dos aplicaciones diferentes, de tal forma que cada una puede ser desarrollada de forma independiente, dando como resultado dos aplicaciones separadas, las cuales pueden ser construidas en tecnologías diferentes, pero siempre respetando el mismo protocolo de comunicación para establecer comunicación.

La idea central de separar al cliente del servidor radica en la idea de centralizar la información y la separación de responsabilidades, por una parte, el servidor será la única entidad que tendrá acceso a los datos y los servirá solo a los clientes del cual el confía, y de esta forma, protegemos la información y la lógica detrás del procesamiento de los datos, además, el servidor puede atender simultáneamente a

varios clientes, por lo que suele ser instalado en un equipo con muchos recursos. Por otro lado, el cliente suele ser instalado en computadoras con bajos recursos, pues desde allí no se procesa nada, simplemente actúa como un visor de los datos y delega las operaciones pesadas al servidor.

Quizás uno de los puntos más característicos de la arquitectura Cliente-Servidor es la centralización de los datos, pues el servidor recibe, procesa y almacena todos los datos provenientes de todos los clientes. Si bien los clientes por lo general solo se conectan a un solo servidor, existen variantes donde hay clientes que se conectan a múltiples servidores para funcionar, tal es el caso de los navegadores, los cuales, para consultar cada página establece una conexión a un servidor diferente.



VENTAJAS:

- **Centralización**: El servidor fungirá como única fuente de la verdad, lo que impide que los clientes conserven información desactualizada.
- **Seguridad**: El servidor por lo general está protegido por firewall o subredes que impiden que los atacantes pueden acceder a la base de datos o los recursos sin pasar por el servidor.
- **Fácil de instalar (cliente)**: El cliente es por lo general una aplicación simple que no tiene dependencias, por lo que es muy fácil de instalar.
- **Separación de responsabilidades**: La arquitectura cliente-servidor permite implementar la lógica de negocio de forma separada del cliente.
- **Portabilidad**: Una de las ventajas de tener dos aplicaciones es que podemos desarrollar cada parte para correr en diferentes plataformas, por ejemplo, el servidor solo en Linux, mientras que el cliente podría ser multiplataforma.

DESVENTAJAS:

- **Actualizaciones (clientes)**: Una de las complicaciones es gestionar las actualizaciones en los clientes, pues puede haber muchos terminales con el cliente instalado y tenemos que asegurar que todas sean actualizadas cuando salga una nueva versión.

- **Concurrencia**: Una cantidad no esperada de usuarios concurrentes puede ser un problema para el servidor, quien tendrá que atender todas las peticiones de forma simultánea, aunque se puede mitigar con una estrategia de escalamiento, siempre será un problema que tendremos que tener presente.

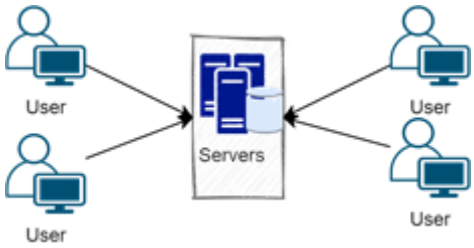
- **Todo o nada**: Si el servidor se cae, todos los clientes quedarán totalmente inoperables.

- **Protocolos de bajo nivel**: Los protocolos más utilizados para establecer comunicación entre el cliente y el servidor suelen ser de bajo nivel, como Sockets, HTTP, RPC, etc. Lo que puede implicar un reto para los desarrolladores.

- **Depuración**: Es difícil analizar un error, pues los clientes están distribuidos en diferentes máquinas, incluso, equipos a los cuales no tenemos acceso, lo que hace complicado recopilar la traza del error.

MODELO DE 2 CAPAS.

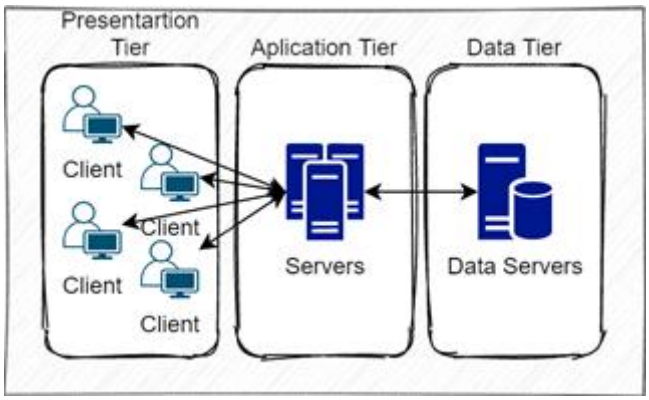
Este tipo de arquitectura utiliza un cliente que ataca los servicios de un servidor donde se ejecuta todo lo necesario para poder dar servicio a las aplicaciones. Es el propio servidor el que, con sus recursos internos, sirve de datos, lógica e información al cliente.



MODELO DE 3 CAPAS

A diferencia del modelo de 2 capas, el servidor al que atacan los clientes hace de intermediario entre otro equipo que almacena los datos, a esta máquina se le conoce como Midelware y suele ser la encargada de proporcionar la lógica de aplicación.

El cliente, simplemente ejecuta la capa de presentación, y el servidor de datos proporciona y registra la información procesada proveniente de la capa de aplicación. Por esta distribución de roles (Cliente/Presentación -> Midelware/Aplicación -> Servidor Datos/Datos) se le conoce como arquitectura de tres niveles.



2. Navegadores.

Un navegador web es un software que permite el acceso a Internet, interpretando la información de distintos tipos de archivos y sitios web para que estos puedan ser visualizados.

La funcionalidad básica de un navegador web es permitir la visualización de documentos de texto, posiblemente con recursos multimedia incrustados. Además, permite visitar páginas web y hacer actividades en ella, es decir, podemos enlazar un sitio con otro, imprimir, enviar y recibir correo, entre otras funcionalidades más.

Los documentos que se muestran en un navegador pueden estar ubicados en la computadora en donde está el usuario, pero también pueden estar en cualquier otro dispositivo que esté conectado en la computadora del usuario o a través de Internet, y que tenga los recursos necesarios para la transmisión de los documentos (un software servidor web).

Tales documentos, comúnmente denominados páginas web, poseen hipervínculos que enlazan una porción de texto o una imagen a otro documento, normalmente relacionado con el texto o la imagen.

El seguimiento de enlaces de una página a otra, ubicada en cualquier computadora conectada a Internet, se llama navegación, de donde se origina el nombre navegador.

Para acceder a estos recursos, se utiliza un identificador único llamado URL (**Uniform Resource Locator**).

El formato general de una URL es “protocolo://máquina/directorio/archivo”.

- Si no se especifica el directorio, toma como directorio el raíz.
- Si no se especifica el fichero, toma alguno de los nombres por defecto. Usualmente estos nombres por defecto son similares a “index.html” o “index.php”.

3. Funcionamiento de los navegadores.

La comunicación entre el servidor web y el navegador se realiza mediante el protocolo HTTP, aunque la mayoría de los navegadores soportan otros protocolos

como FTP y HTTPS (una versión cifrada de HTTP basada en Secure Socket Layer o Capa de Conexión Segura (SSL)).

La función principal del navegador es obtener documentos HTML e interpretarlos para mostrarlos en pantalla. En la actualidad, no solamente descargan este tipo de documentos sino que muestran con el documento sus imágenes, sonidos e incluso vídeos streaming en diferentes formatos y protocolos. Además, permiten almacenar la información en el disco o crear marcadores (bookmarks) de las páginas más visitadas.

Algunos de los navegadores web más populares se incluyen en lo que se denomina una Suite. Estas Suite disponen de varios programas integrados para leer noticias de Usenet y correo electrónico mediante los protocolos NNTP, IMAP y POP.

Los primeros navegadores web sólo soportaban una versión muy simple de HTML. El rápido desarrollo de los navegadores web propietarios condujo al desarrollo de dialectos no estándares de HTML y a problemas de interoperabilidad en la web. Los más modernos (como Google Chrome, Mozilla, Netscape, Opera e Internet Explorer / Microsoft Edge) soportan los estándares HTML y XHTML (comenzando con HTML 4.01, los cuales deberían visualizarse de la misma manera en todos ellos).

Los estándares web son un conjunto de recomendaciones dadas por el World Wide Web consortium (W3C) y otras organizaciones internacionales acerca de cómo crear e interpretar documentos basados en la web. Su objetivo es crear una web que trabaje mejor para todos, con sitios accesibles a más personas y que funcionen en cualquier dispositivo de acceso a Internet.

LENGUAJES INTERPRETADOS.

Como ya sabemos, un lenguaje **compilado**, es aquel que, a partir de su código fuente, mediante el proceso de compilación se genera un fichero ejecutable

En nuestro modulo vamos a trabajar con JavaScript. JavaScript es un lenguaje del lado del cliente **interpretado** línea a línea por el navegador, mientras se carga la página, que solamente es capaz de realizar las acciones programadas en el entorno de esa página HTML donde reside. Sólo es posible utilizarlo con otro programa que sea capaz de interpretarlo, como los navegadores web.

PRINCIPALES NAVEGADORES

- **Microsoft Edge (Antiguo Internet Explorer)**
  - URL Oficial: https://www.microsoft.com/es-es/windows/microsoft-edge
  - Sustituye al antiguo “Internet Explorer”.
  - Microsoft Edge está diseñado para ser un navegador web ligero con un motor de renderizado de código abierto construido en torno a los estándares web.

- **Mozilla Firefox**
  - URL Oficial: https://www.mozilla.org/es-ES/firefox/new/
  - Mozilla Firefox es un navegador web libre y de código abierto desarrollado por la Fundación Mozilla. Usa el motor Gecko para renderizar páginas webs, el cual implementa actuales y futuros estándares web.
    - Posee una versión para desarrolladores: “Firefox Developer Edition” https://www.mozilla.org/es-ES/firefox/developer/

- **Google Chrome**
  - URL Oficial: https://www.google.com/chrome/
  - Google Chrome es un navegador web desarrollado por Google y compilado con base en varios componentes e infraestructuras de desarrollo de aplicaciones (frameworks) de código abierto, como el motor de renderizado Blink (fork de WebKit). Está disponible gratuitamente bajo condiciones específicas del software privativo.

- **Safari**
  - URL oficial: http://www.apple.com/es/safari
  - Safari es un navegador web de código cerrado desarrollado por Apple Inc. Está disponible para Mac OS, iOS (el sistema usado por el iPhone, el iPod touch y iPad) y Windows (sin soporte desde el 2012).

- **Opera**
  - URL oficial: http://www.opera.com/es
  - Opera es un navegador web creado por la empresa noruega Opera Software. Usa el motor de renderizado Blink. Tiene versiones para escritorio, teléfono y tablet.

4. Compatibilidad con navegadores Web.

Si has probado usar diferentes navegadores de Internet para visitar tus sitios preferidos, posiblemente hayas notado que los mismos sitios online parecen algo diferentes dependiendo del navegador que estés usando Esto es porque el despliegue de la página varía según la compatibilidad que tenga el navegador. La compatibilidad en los navegadores de Internet es la habilidad que tiene para interpretar correctamente el lenguaje HTML que muestra las páginas Web. Lo que ocurre es que este lenguaje es “entendido” de una forma ligeramente diferente por cada navegador.

Muchos sitios están diseñados para mostrarse bien en el popular Internet Explorer ya que se supone que es el que más gente utiliza. Sin embargo, si eres un diseñador Web tu tarea debería ser codificar un sitio para una compatibilidad total para que las páginas aparezcan adecuadamente en otros navegadores, como Firefox, Opera, etc.

Esto es algo que levanta más de un dolor de cabeza entre los administrares Web. Con nuevos lenguajes de scripting para páginas Web la cosa se complica más aun, ya que viejos navegadores pueden no entender los códigos nuevos.

Por razones de seguridad, mucha gente deshabilita los lenguajes de scripting en sus navegadores, lo cual reduce esta compatibilidad. Si el administrador Web ha sido cuidadoso, debería haber una manera de navegar por el sitio los nuevos códigos que se van implementando. Por desgracia no suele ser el caso, y un navegador con el scripting y las cookies desactivadas puede no ser capaz de entrar en un sitio online.

Para solucionar el problema, muchos internautas mantienen dos o tres navegadores en su ordenador. El que viene instalado por defecto en Windows es el Internet Explorer, pero cada vez más gente decide instalar también Firefox y Google Chrome. Si ven que están teniendo problemas con un navegador, simplemente pueden abrir otro y probar si se solucionar el problema.

Se puede incluso dejar un navegador con scripting activado y otro sin activar. Uno podría ser para sitios de confianza y otro para páginas Web que no conocemos demasiado bien. Esta es una de las mejores soluciones que hay por el momento.

Aparte de la diferencia que hay entre los navegadores de Internet, otro factor que entra en juego es si el usuario está utilizando una plataforma y otra en su ordenador. Dependiendo de si se está usando Windows, Linux y otro sistema operativo, el mismo navegador puede mostrar las páginas algo diferentes en cada una de las plataformas.

Por este motivo, si vas a contratar a un diseñador para que haga tu sitio Web, le debes preguntar por estos temas de compatibilidad. Mientras que muchos usuarios tienen más de un navegador instalado en su equipos, se tiene que asumir que la mayoría de la gente abandonará un sitio si no lo ve correcto o con anomalías.

5. Lenguajes de programación en entorno cliente.

Antes de enumerar los distintos lenguajes de programación que podemos encontrar en entornos clientes vamos a definir qué significa eso de entornos de desarrollo del lado del cliente y del lado del servidor.

¿Qué significa 'LADO DEL CLIENTE'?

En el desarrollo web, el "lado del cliente" hace referencia a todo lo que en una aplicación web se muestra o tiene lugar en el cliente (dispositivo del usuario final). Esto incluye lo que ve el usuario, como texto, imágenes y el resto de la interfaz de usuario, junto con cualquier acción que una aplicación lleve a cabo en el navegador del usuario.

¿Qué significa 'LADO DEL SERVIDOR'?

Como ocurre con el lado del cliente, "lado del servidor" se refiere a todo lo que ocurre en el servidor, en lugar de en el cliente. En el pasado, casi toda la lógica empresarial se ejecutaba en el lado del servidor, y esto incluía la representación de páginas web dinámicas, la interacción con las bases de datos, la autenticación de identidades y las notificaciones push.

Aclarado estos conceptos, pasamos a enumerar los lenguajes de entorno cliente más usados actualmente.

- React
- JavaScript
- CSS
- HTML
- Angular
- Vue
- jQuery
- Swift
- SASS
- Elm

6. Características de los lenguajes de script. Ventajas y desventajas sobre la programación tradicional.

Un lenguaje Script tiene una serie de características básicas:

- **Son interpretados**. El código de un script no necesita ser compilado, basta con que sea interpretado por un programa para que se ejecute su código. Por ejemplo, un navegador web es un intérprete que puede ejecutar un script escrito en JavaScript (incorporan programas para interpretar como Chakra Core, que lo utiliza Microsoft en su navegador web).

- **No necesitan la declaración de variables**. Estos lenguajes no necesitan una declaración explícita de variables para asignar su tipo. El tipo de dato de cada variable es deducido del contexto en el que se utiliza, en el momento en el que se le asigna un valor. Esta tipología débil requiere de una especial atención a la hora de trabajar con variables.

- **Utilizan instrucciones**. Las instrucciones son fundamentales para que el intérprete pueda ejecutar las acciones que se quieren realizar.

- **Código simple**. Los scripts pueden llegar a ser complicados, pero normalmente se trata de programas cortos y concisos que persiguen ejecutar una acción concreta, por lo que se busca la simplicidad y la eficiencia.

VENTAJAS DE USO.

- **Velocidad**: Tienden a ser muy rápidos porque a menudo se ejecuta inmediatamente en el navegador. Entonces mientras no requiera recursos externos, JavaScript no tiene permitido retrasarse por llamados del servidor backend.

- **Simplicidad**: La sintaxis de JavaScript está inspirada por Java y es relativamente sencillo de aprender comparado a otros lenguajes de programación populares como C++.

- **Popularidad:** JavaScript esta por todas partes de la web, y con la llegada de Node.js, se ha incrementado su uso en backend. Hay incontables recursos para aprender JavaScript. Tanto StackOverflow como GitHub muestran un creciente número de proyectos que usan JavaScript, y la popularidad que ha alcanzado en los recientes años se espera que siga creciendo.

- **Compatibilidad:** A diferencia de PHP u otros lenguajes scripting, JavaScript puede ser usado en cualquier página web. JavaScript puede ser usado en diferentes tipos de aplicaciones gracias al soporte en otros lenguajes como Pearl y PHP.

- **Server Load**: JavaScript es client-side, entonces esto reduce la demanda de servidores en general, y las aplicaciones sencillas puede que no necesiten el servidor del todo.

- **Interfaces sencillas**: JavaScript puede ser usado para crear características como arrastrar y soltar, y componentes tales como las diapositivas, lo cual mejora enormemente la interfaz de usuario y la experiencia del sitio.

- **Funcionalidad extendida**: Los desarrolladores pueden extender la funcionalidad de las páginas web mediante fragmentos de JavaScript para un tercer grupo de extensiones como Greasemonkey.

- **Versatilidad**: Hay muchos métodos para usar JavaScript mediante servidores Node.js. Si tú estas en bootstrap Node.js con Express, puedes usar un documento database como MongoDB, y usar JavaScript en el frontend para clientes. Es posible desarrollar una aplicación completa de JavaScript desde principio a fin solo usando JavaScript.

- **Actualizaciones**: Desde que la llegada de ECMAScript 5 (la especificación escrita en que se basa JavaScript), ECMA International se ha dedicado a actualizar JavaScript anualmente. Hasta el momento, hemos recibido soporte para el navegador ES6 en 2017 y esperamos que ES7 sea soportado en un futuro.

DESVENTAJAS DE USO.

- **Seguridad Client-Side**: Desde que el código en JavaScript es ejecutado en el client-side, bugs y descuidos pueden ser explotados algunas veces para malos propósitos. Por esto, algunas personas deciden desactivar JavaScript por completo.

- **Soporte del navegador**: Mientras server-side script siempre produce el mismo resultado, algunas veces diferentes navegadores interpretan el código JavaScript de manera distinta. Estos días las diferencias son mínimas, y no deberías tener que preocuparte mientras compruebes tu código en la mayoría de los navegadores.

7. Tecnologías y lenguajes asociados.

Como se ha comentado en el punto 5, lenguajes de entorno cliente más usados actualmente.

- React
- JavaScript
- CSS
- HTML
- Angular
- Vue
- jQuery
- Swift
- SASS
- Elm

8. Integración del código con las etiquetas HTML.

Si quieres mostrar contenido estático como, por ejemplo, un conjunto de imágenes, entonces HTML puede hacer el trabajo por ti. Sin embargo, las páginas estáticas son cosa del pasado.

La mayor parte del contenido actual es interactivo e incluye vistosas presentaciones de diapositivas, formularios y menús. Estos elementos mejoran la experiencia del usuario y añaden dinamismo al sitio web.

Para ello se utilizan lenguajes de scripting y JavaScript es uno de los más famosos en este sentido. Con él, los desarrolladores pueden crear sitios web que interactúan con el usuario y viceversa. Aunque hay muchos otros lenguajes disponibles, ninguno es tan popular como JavaScript. Para aprovecharlo al máximo, se utiliza junto con HTML.

Hay dos formas de insertar JavaScript en HTML y hacer que funcionen juntos. Echemos un vistazo a algunas de las formas en que podemos conectar JavaScript con HTML.

MEDIANTE ETIQUETA <SCRIPT></SCRIPT>

La primera forma de insertar JavaScript en HTML es directa. Puedes hacerlo utilizando la etiqueta <script> </script> que debe envolver todo el código JavaScript que escribas. Se puede agregar el código JavaScript:

- Entre las etiquetas <head>
- Entre las etiquetas <body>

Dependiendo de dónde agregues el código JavaScript en tu archivo HTML, la carga será diferente. Por lo general se recomienda agregarlo en la sección <head> para que permanezca separado del contenido de tu archivo HTML. Pero colocarlo dentro de <body> puede ayudar a mejorar la velocidad de carga, ya que el contenido del sitio web se cargará más rápido, y solo después de eso se procesará el JavaScript.

ENLAZANDO CON FICHERO .JS.

A veces, insertar JavaScript en HTML directamente no parece ser la mejor opción. Sobre todo porque algunos scripts de JS deben utilizarse en varias páginas, por lo tanto, es mejor mantener el código JavaScript en archivos separados. Es por esto que la forma más aceptable de incluir JavaScript en HTML es importando archivos externos.

Se puede hacer referencia a estos archivos desde adentro de los documentos HTML, al igual que hacemos referencia a los documentos CSS. Algunos de los beneficios de agregar código JavaScript en archivos separados son:

Cuando los códigos HTML y JavaScript están separados, se cumple el principio de diseño de separación y hace que todo sea mucho más sostenible y reutilizable.

La legibilidad y el mantenimiento del código es mucho más fácil.

Los archivos JavaScript en caché mejoran el rendimiento general del sitio web al disminuir el tiempo que tardan las páginas en cargarse.

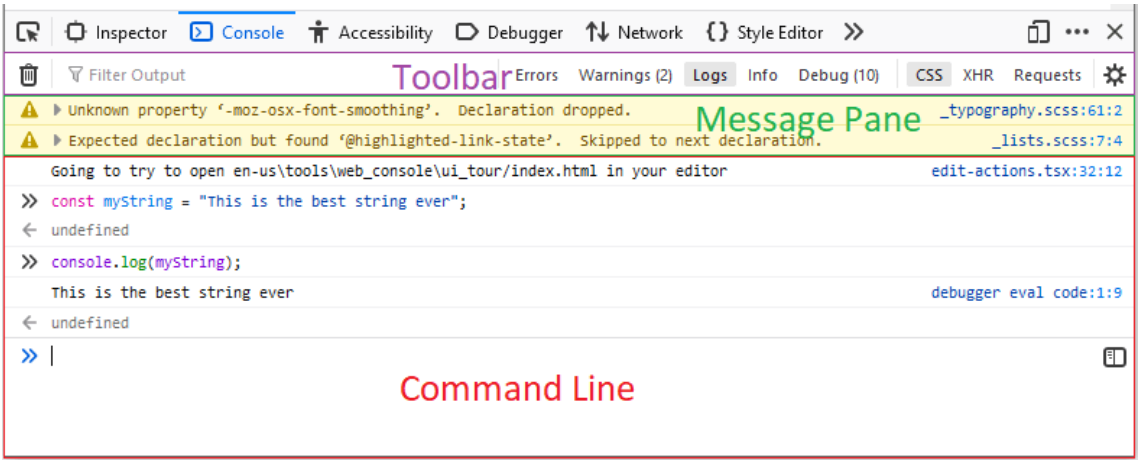
Podemos hacer referencia al archivo JavaScript desde HTML así:

```
<script src="js/myscript.js"></script>
```

9. Herramientas de programación.

La mayoría de los navegadores incorporan de manera nativa herramientas para facilitar el desarrollo, entra la que destacamos la “Consola Web”. Asimismo, también mediante ampliaciones (extensiones, plugins, etc.) se amplían características para facilitar el desarrollo y la depuración de código.

CONSOLA WEB.



Esta consola incluye varias pestañas:

- Red: registro de Peticiones HTTP.
- CSS: análisis y errores CSS.
- JS: análisis y errores Javascript
- Seguridad: registra advertencias o fallos de seguridad.
- Registro: registra mensajes enviados al objeto “window.console”
- Servidor: registrar mensajes recibidos del servidor Web.

Para más información del uso de la “Web console” de Mozilla Firefox en la web https://firefox-source-docs.mozilla.org/devtools-user/web_console/index.html

ENTORNOS DE DESARROLLO.

Existen diversos entornos de desarrollo, Desde los más sencillos (Brackets, Notepad++, Sublime, Visual Studio Code, etc...) a interfaces más complejas (Aptana, Eclipse, Visual studio, etc...).

Recomendamos el uso de **Visual Studio Code**. Es muy potente y posee un gran ecosistema de plugins para ampliar su funcionalidad. https://code.visualstudio.com/